CalcDb: the Access database engine, rebuilt

Draft 1 · 2026-11-10 · working document for review · CHARTER.md has the detail

What it is

CalcDb is the database engine of DNA Calc: a clean-room Rust reimplementation of Jet 4 and ACE, with the DAO and ADO object models on top. VBA, `.xll` and .NET add-ins that use ACE in Excel today run unchanged inside DNA Calc against CalcDb. Files go both ways: Access opens what CalcDb writes, Compact & Repair finds nothing to fix, and anything CalcDb does not understand survives a rewrite byte for byte. The core also runs off Windows, including in DNA Calc's browser build.

We are not designing a database; we are rebuilding one whose behaviour is fixed by twenty-five years of files, queries and add-ins. Success is how often we match ACE, not what we invent.

In and out

- **In:** Jet 4 `.mdb` and ACE `.accdb`; all Access SQL, including crosstabs, `PARAMETERS`, `DISTINCTROW`, action queries, DDL and both wildcard modes; QueryDefs read and written in Access's own catalog form; ACE's results, types, Null handling, sort orders and error numbers; full DAO first, then ADO from VBA (including `Microsoft.ACE.OLEDB.12.0` strings); COM on x64 with DAO's real interface identities, side by side with ACE.
- **Later:** sharing a live file with Access and ACE processes through the lock file; encryption, linked tables, complex fields, our own compact and repair, OLE DB and ODBC.
- **Out:** running the Access front end (forms, reports, macros and modules are preserved and readable, never run), Jet 3, 32-bit and ARM64 COM, server roles, and any SQL dialect of our own.
- **Sources:** public documents, Jackcess (Apache 2.0, ported with a licence ledger), mdbtools (facts only) and observed Access/ACE behaviour; never decompiling. FrankenSqlite and asupersync only through clean-room notes, reviewed, with provenance.

Principles

1. **ACE is the specification; the oracle is the judge.** Every behaviour is settled by asking real ACE and Access on Windows; answers are kept as captures keyed by input. A differential harness runs the same SQL, DAO calls and file operations against both engines and compares rows, types, errors and bytes. A refusal is a failure in every count. The corpus of files written by many Access versions is a product in its own right.
2. **ISAM first, SQL second.** Jet is a cursor engine with SQL on top, and DAO code leans on `Seek`, bookmarks and table-type recordsets. Tables, indexes, cursors and transactions form one layer, shared by DAO, ADO and SQL.
3. **The file is the truth.** No side format and no write-ahead log of our own: every commit leaves a valid Access file, so write ordering is designed, not incidental. Index keys are byte-identical to ACE's, including legacy sort orders, or Access reports damage. What we do not model we carry as opaque bytes.
4. **Describe the format once.** Page layouts, table definitions, usage maps and catalog rows are written down as data. Reader, writer, validator, fuzzer and a page inspector are all driven from that one description, so they cannot drift apart; each rule cites the observation or source it came from.
5. **One value model, one evaluator.** Values, conversions and the VBA function library come from CalcVBA, so a field read by a query, passed to `Format` and handed back to VBA behaves identically everywhere. ACE's sandbox rules apply. Host functions are an explicit opt-in that marks a query as not portable to Access.
6. **Fast alone, then together.** Most Access work is one user, one process. FrankenSqlite's experience is that multi-version machinery taxes the single-writer path heavily and that disjoint writers still collide on shared tree pages and the allocator. So exclusive mode is the fast path, measured first; in-process snapshots and then the ACE lock-file protocol follow, behind a locking interface that exists from day one.
7. **Deterministic by construction.** Time, I/O, scheduling and randomness reach the engine only through injected interfaces. A deterministic simulator replays any interleaving from a seed, injects faults, and crashes the engine at every write point to check the file afterwards. Concurrency bugs are found by search, not by luck.
8. **Synchronous core, owned work.** Object-model calls run on the caller's thread, as VBA expects; the core needs no async runtime. Background work (read-ahead, flushing, cleanup) has an owner, a budget and a cancellation protocol of request, drain and finalize. Locks, pins and open cursors are obligations settled when their owner closes. No user code, VBA functions included, ever runs while the engine holds an internal lock.
9. **Portable core, thin shells.** The format, ISAM, catalog, expression and SQL crates do no I/O and build for wasm32. Native files, the lock file and COM are thin crates around them, and the native Rust API used by CalcVBA is as complete as COM.
10. **Explainable and measured.** Plans are inspectable, the plan shown is the plan run, and tests assert plan shape (seek versus scan) as well as results. Performance work has budgets per path, noise control and a ledger of what failed. A ledger built from oracle results shows which DAO members, SQL features and types match ACE.
11. **Evidence over ceremony.** Working code and clear tests beat process apparatus. One of everything: one format description, one ISAM, one evaluator, one oracle, one harness.

Next

Order of work: Oracle, corpus and read path → exclusive writing → `SELECT` and expressions → action queries, DDL, QueryDefs → DAO (native, COM, .NET) → ADO → in-process sharing → sharing with ACE → the rest. **Open:** home of the shared value-model crate; in-process COM activation for add-ins; reuse of CalcVBA's JIT for expressions; a private undo file for exclusive-mode crash safety; the oracle's ACE and Access builds.